FOL: A Functional Object Lisp

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

Abstract

We present FOL (Functional Object Lisp), a Lisp dialect combining Clojure's persistent data structures with CLOS-style object orientation. We find that persistent objects and CLOS are not antagonistic but synergistic: the combination yields metaobject-protocol-enabled versioned objects, declarative event sourcing via method combinations, and content-based observable notifications via predicate dispatch—patterns more natural than in either parent alone. FOL transpiles to Common Lisp; benchmarks show 2–8× overhead for feature-specific workloads and 3–11× for a digital logic simulator, with the gap narrowing at scale. A parallel variant using `preduce` achieves a 3.0× ratio on a 1024-gate pipeline. FOL achieves 13–38% reductions in source lines of code across benchmarks. The implementation runs atop FSet and Sycamore persistent data structure libraries, providing Clojure-compatible persistent collections and a meta-object protocol adapted for immutable storage. Objects use a hybrid layout—native CLOS slots for the first 32 fields, FSet persistent vectors for overflow—achieving native access speed for common small objects.

CCS Concepts

• **Software and its engineering** → **Functional languages**; **Object oriented languages**; *Extensible languages.*

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

1 Introduction

FOL explores the synergy between Common Lisp-style object orientation (CLOS) and Clojure-style immutable data structures. While CLOS offers powerful multiple dispatch, method combinations, and a metaobject protocol (MOP), it is fundamentally rooted in mutable state. Conversely, Clojure's immutable model provides $O(\log n)$ updates and structural sharing but lacks the introspective extensibility of CLOS. FOL merges these paradigms, demonstrating that persistent objects and CLOS are not only compatible but synergistic.

Immutable objects simplify concurrent development by eliminating synchronization conflicts and manual locking. FOL implements these objects by adapting the CLOS MOP to use immutable data structures for slot storage. We implemented FOL as a standalone Lisp rather than a library to ensure language-level correctness; a library-based approach would struggle to enforce persistence invariants and manage the integration of persistent and primitive types.

The combination of immutability and CLOS enables patterns more natural than in either parent alone:

- **Versioned objects via the MOP**: Functional updates transparently produce instances of the new schema while preserving old snapshots, trading CLOS's explicit migration hooks for corruption safety.
- **Event sourcing via method combinations**: Declarative `:around` methods intercept all mutations to build immutable audit logs automatically, a guarantee harder to enforce in Clojure.
- **Observable notifications via predicate dispatch**: Content-based routing of change events without manual `cond` cascades, combining persistent updates with extensible generic functions.

Beyond architectural synergy, FOL provides concise predicate dispatch and enhanced destructuring. Our benchmarks show that FOL achieves a 13–38% reduction in SLOC while maintaining acceptable performance for workloads that benefit from immutability guarantees.

2 Language Design

2.1 Core Philosophy

FOL makes several design commitments:

(1) **Immutability by default**: All objects and collections (except streams, atoms, etc.) are persistent—in the sense of Driscoll et al. [7], preserving previous versions after modification, not database persistence.
(2) **Structural sharing**: Updates create new versions sharing structure with old.
(3) **Object identity**: Distinguishes version identity (`eq`) from value equality (`=`).
(4) **Generic functions**: Multiple dispatch over destructuring patterns.
(5) **Meta-object protocol**: Introspection and extension via adapted MOP protocols.

2.2 Identity and Equality

Following Hickey's epochal time model [12], FOL simplifies Common Lisp's four-level equality hierarchy (`eq`/`eql`/`equal`/`equalp`) to two levels. `eq` tests *reference identity*—each functional update produces a new object, so `(eq alice older-alice)` returns false even when both represent "Alice." FOL's `=` tests *value equality* via generic dispatch: numbers compare with numeric coercion, persistent objects compare by structural comparison of storage maps, and strings compare by content—subsuming the roles of `eql`, `equal`, and `equalp`. Two independently constructed objects with identical slot values are `=` and may be used as map keys.

2.3 Syntax and Readability

FOL adopts Clojure's literal and definition syntax for conciseness. Persistent collections use literal syntax with Clojure semantics:

```
[1 2 3]                 ; Vectors
{:name "Alice" :age 30}  ; Maps
#{1 2 3 4}              ; Sets

(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])

(defn summarize [{:keys [name age]}]
  (str name " is " age " years old"))
```

Type names use Dylan-style angle brackets (`<person>`).

2.4 Persistent Object Protocol

By default, user-defined classes inherit from `<persistent-object>`:

```
(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])

(def alice (make <person> :name "Alice" :age 30))
(def older-alice (assoc alice :age 31))
(:age alice)       ; => 30 (unchanged)
(:age older-alice) ; => 31
```

`defclass` supports standard CLOS options (`:type`, `:initarg`, `:initform`, `:reader`). Keywords in function position act as accessor functions (Clojure-style).

Slot storage uses a hybrid strategy inspired by V8's hidden classes [4]: objects with ≤ 32 slots store all values in native CLOS slots, providing native memory access speed; objects with > 32 slots store the first 32 in native CLOS slots and overflow the remainder into an FSet persistent vector. Functional updates (`assoc`) shallow-copy the native slots and, for wide objects, produce a new FSet vector via structural sharing. This split threshold matches the branching factor of FOL's persistent vectors, keeping updates $O(1)$ for common small objects and $O(\log n)$ only for the overflow portion.

3 Architecture and Implementation

3.1 Collection Implementation

FOL wraps FSet [5] and Sycamore data structures for its collections:

- **Vectors** (`[1 2 3]`): 32-way branching trees supporting $O(\log_{32} n)$ random access.
- **Maps** (`{:a 1 :b 2}`): Hash array mapped tries (HAMTs) with efficient key-value operations.
- **Sets** (`#{1 2 3}`): Weight-balanced binary trees for ordered iteration.
- **Lists**: Persistent cons cells with $O(1)$ prepend and $O(n)$ access.

FOL also provides thirteen additional collection types from `<array-dict>` to `<dense-int-set>` to `<array>`. These have been optimized to use the combination of FSet and Sycamore structures that provides the best performance at both small sizes (<20 elements) and at scale (>1000 elements), or have been hand-written for collections not based on persistent structures (list, lazy-seq).

3.2 Runtime Representations

Most runtime representations other than collections and user-defined objects utilize a Common Lisp base. Primitives (`<bool>`, `<string>`, the numeric tower, `<symbol>`) are unwrapped CL values; functions with their closures are CL functions; the package system and error handling derive from CL. Garbage collection is inherited—old versions of user-defined objects are collected when unreferenced; structural sharing ensures only unshared portions are reclaimed.

3.3 MOP Protocol Adaptation

FOL's persistent metaclass adapts rather than replaces the MOP. Table 1 summarizes the protocol disposition.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
`slot-value-using-class`	Standard	Native CLOS slot read
`slot-missing`	Extended	Overflow to FSet vector
`(setf slot-value-...)`	Guarded	Blocks post-init writes
`slot-makunbound-...`	Blocked	Immutability invariant
`validate-superclass`	Extended	Cross-metaclass mixing
`initialize-instance`	Extended	Populates map from initargs
`update-instance-...`	Omitted	Lazy migration (Sec. 5.4)
`change-class`	Omitted	Violates immutability

Introspection protocols (`class-slots`, `class-precedence-list`, `slot-definition-*`) work unchanged because FOL's class *structure* is standard CLOS—only slot *mutation* is guarded. For objects with ≤ 32 slots, `slot-value-using-class` uses standard CLOS slot reads at native speed; the `setf` method guards against post-initialization writes. For wide objects (> 32 slots), overflow slots are intercepted by `slot-missing` and redirected to the FSet persistent vector. The `validate-superclass` extension permits cross-metaclass inheritance (persistent classes may inherit from standard classes). This adaptation requires the MOP: persistence cannot be implemented as a library because transparent mutation guarding and overflow routing demand metaclass-level interception.

4 Features

4.1 Clojure Features

FOL implements key Clojure abstractions:

- **Transducers** [13]: Composable transformation pipelines separating the "what" (map, filter, take) from the "how" (eager, lazy, parallel).
- **Lazy sequences**: On-demand computation enabling infinite sequences and efficient streaming.
- **Atoms, Refs/STM, and Agents**: Clojure's concurrency primitives for managing mutable state in a thread-safe manner.
- **Sequence functions**: Over 100 functions including `map`, `filter`, `reduce`, `take`, `drop`, `partition`, `group-by`, and `frequencies`.
- **Macros**: Clojure-style `defmacro` with syntax-quote, unquote, and splicing unquote.

4.2 Generic Function Integration

FOL's generic functions dispatch by arity first, then by type predicates and specificity within an arity group. Methods at different arities may coexist on the same generic:

```
(defgeneric process [x])
(defmethod process
  ([(x <vector>)] (vec (map process x)))
  ([(x <dict>)]   (update-vals x process))
  ([(x <number>)] (* x 2)))

(process [1 {:a 2 :b 3} [[4]]])  ; => [2 {:a 4 :b 6} [[8]]]
```

Multi-clause `defmethod` forms expand to separate method definitions. The `:before`, `:after`, and `:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

`defn` vs `defgeneric`/`defmethod`: `defn` defines a standalone multi-clause function with predicate dispatch and specificity ordering; it does not create a generic function. `defgeneric` declares a generic function; `defmethod` adds methods to it. Only `defmethod` accepts qualifiers. Both `defn` and `defmethod` support multi-clause predicate dispatch.

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

4.3 Predicate Dispatch

FOL extends pattern matching with predicate specializers. The syntax `(var (fn arg0 arg1 ...))` evaluates `(fn var arg0 arg1 ...)` at runtime—the bound value is inserted as the first argument. Predicates work with any function—equality, comparison, type tests, or user-defined:

```
;; Range checking with comparison predicates
(defn age-group
  ([(age (< 13))] :child)
  ([(age (< 20))] :teen)
  ([(age (< 65))] :adult)
  ([age] :senior))

;; Type predicates
(defn process-value
  ([(x (<number>?))] (* x 2))
  ([(x (<string>?))] (str x x))
  ([(x (<vector>?))] (vec (map process-value x)))
  ([x] x))
```

4.3.1 Predicate Specificity. FOL orders patterns by *constraint strength*—how narrowly a pattern constrains its accepted values. More specific patterns are tested first, ensuring predictable dispatch regardless of definition order:

Table 2: Predicate Specificity Levels

Level	Pattern Type	Constraint	Example
3	Predicate	Single value/narrow range	`(n (= 5))`
2	Type	All instances of a type	`(x <number>)`
1	Destructuring	Structural shape only	`[a b c]`
0	Any	Universal match	`x, _`

This ordering is a deliberate heuristic chosen for predictability rather than full logical subsumption, which is undecidable in general [9]. Within a specificity level, definition order is preserved, giving programmers explicit control over tie-breaking. For sequence patterns, specificity extends to element-level comparisons: `[(x (< 0)) y]` is more specific than `[(x <number>) y]` because its first element has higher specificity.

4.3.2 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using inference rules. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p.

$$\textsc{Spec-Level}\ \frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2} \qquad \textsc{Spec-Seq-Head}\ \frac{p_1 \prec q_1 \quad n \geq 1}{[p_1 \ldots p_n] \prec [q_1 \ldots q_n]}$$

$$\textsc{Spec-Seq-Tail}\ \frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2 \ldots] \prec [q_2 \ldots]}{[p_1 \ldots p_n] \prec [q_1 \ldots q_n]}$$

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $idx(p)$ denote the definition index of the method clause containing p.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable under $\prec$; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$ within it. This lexicographic combination ensures deterministic dispatch: arity-first, then specificity, then definition-order.

4.3.3 Dispatch Semantics. Predicate specializers work in `defn`, `defmethod`, and anonymous `fn` forms but not in macro parameter lists (`defmacro` rejects type and predicate specializers). Dispatch is $O(N)$ in the number of patterns at a given arity, with patterns pre-sorted by specificity at definition time. FOL's type system is entirely dynamic—type specializers are checked at runtime. The boundary between type and predicate categories is conventional: type predicates like `(<number>?)` are recognized syntactically and assigned level 2, while other predicates (including user-defined type tests) receive level 3. This means `(x (<number>?))` and `(x <number>)` accept the same values but occupy different specificity levels if mixed—in practice this is benign, as programmers use one form or the other.

FOL's dispatch maintains three invariants: (1) **Exhaustive matching**—unmatched calls signal `no-matching-clause` rather than silently failing; (2) **Deterministic dispatch**—the total order $\prec_{\text{disp}}$ ensures exactly one clause matches for any input; (3) **Type predicate consistency**—type specializers use the same predicates as explicit checks, so `(x <number>)` matches exactly the values for which `(<number>? x)` returns true.

5 Synergy in Practice

This section demonstrates patterns that are *more natural or more concise* in FOL than in either Clojure or standard CLOS alone, because they exploit the combination of persistent data, CLOS method combinations, and predicate dispatch. Full source for all examples is available in the repository: FOL code in `fol-code/`, Common Lisp equivalents in `lisp-code/`.

defn	::=	`(defn` *name* (*single-clause* | *multi-clause*))	
defgeneric	::=	`(defgeneric` *name* (*single-pattern* | *multi-pattern*))	
defmethod	::=	`(defmethod` *name qualifier*? (*single-clause* | *multi-clause*))	
fn	::=	`(fn` (*single-clause* | *multi-clause*))	
single-clause	::=	*pattern body*$^+$	
multi-clause	::=	(*pattern body*$^+$)$^+$	
single-pattern	::=	*pattern*	
multi-pattern	::=	*pattern*$^+$	
pattern	::=	`[`*param**`]` | `[`*param** & *rest-param*`]`	
param	::=	*symbol*	(simple binding)
	|	(*symbol type*)	(type specializer)
	|	(*symbol* (*fn-name arg**))	(predicate specializer)
	|	*destructure*	(nested destructuring)
destructure	::=	`[`*param**`]`	(sequential)
	|	`{:keys [`*key-spec**`]}`	(map)
key-spec	::=	*symbol* | (*symbol type*) | (*symbol pred*)	
type	::=	<*type-name*>	
qualifier	::=	`:before` | `:after` | `:around`	

Figure 1: FOL Multi-Pattern Dispatch Grammar (simplified)

5.1 Trade Compliance

We implement a trade compliance validator in FOL and plain Common Lisp (repository files `compliance.fol`, `compliance.lisp`). The FOL version uses concise predicate functions, persistent objects, and collection literals; the CL version uses explicit class definitions, a manual `cond` cascade, and package boilerplate. Table 3 compares the implementations.

```
(defclass <trade> []
  [[symbol :initarg :symbol :accessor trade-symbol]
   [amount :initarg :amount :accessor trade-amount]
   [price  :initarg :price  :accessor trade-price]
   [side   :initarg :side   :accessor trade-side]])

(defn restricted-symbol? [trade]
  (contains? #{:AMZN :GOOG :META :MSFT}
             (trade-symbol trade)))

(defn high-value? [trade]
  (> (* (trade-price trade) (trade-amount trade))
     1000000.00))

(defn validate-trade [trd]
  (cond
    ((restricted-symbol? trd)
     (make '<list> :status :rejected
           :reason (str "Symbol " (trade-symbol trd)
                        " is on the restricted list")))
    ((high-value? trd)
     (make '<list> :status :manual-review
           :reason "Trade value exceeds $1M limit"))
    (:else
     (make '<list> :status :approved
           :id (gensym "TRD")))))
```

The 17% SLOC reduction comes primarily from FOL's concise `defclass` syntax, set literals (`#{...}`), and elimination of package boilerplate. The CL version requires a separate `defpackage` declaration, an explicit `make-instance` constructor wrapper, and `defparameter` for the restricted symbol list. The predicate logic is slightly *larger* in FOL because persistent map construction (`make '<list> ...`) is more verbose than CL's native property lists. The advantage here is not brevity but *immutability*: each validation result is a persistent map that cannot be mutated after creation.

Table 3: Code Comparison—Trade Compliance

Category	FOL	CL	Savings
Class definition	5	9	44%
Predicates / logic	16	14	−14%
Dispatch / validation	12	14	14%
Module boilerplate	2	10	80%
Test harness	15	13	−15%
Total SLOC	**50**	**60**	**17%**

5.2 Event Sourcing with Method Combinations

FOL enables event sourcing by combining `:around` methods with persistent event logs. The key mechanism is a single `:around` method that intercepts all command dispatches:

```
(defclass <account> [<persistent-object>]
  [[balance :initform 0]
   [events  :initform []]])

(defmethod apply-command :around [agg cmd]
  (bind [result (call-next-method)
         event  {:command cmd :timestamp (now)}]
    (assoc result :events
           (conj (get result :events) event))))

(defmethod apply-command
  ([agg (cmd (deposit?))]
   (assoc agg :balance
          (+ (get agg :balance) (get cmd :amount))))
  ([agg (cmd (withdraw?))]
   (assoc agg :balance
          (- (get agg :balance) (get cmd :amount)))))
```

This `:around` method applies automatically to every `apply-command` invocation, including future methods added after deployment. The

Clojure equivalent uses `defmulti`/`defmethod` but lacks `:around` methods, so event logging requires a manual wrapper that callers must remember to invoke. Table 4 compares the implementations.

Table 4: Code Comparison—Event Sourcing

Category	FOL	CL	Savings
Class / factory	3	6	50%
Event logging	5	8	38%
Command methods	10	14	29%
Replay / bench	10	17	41%
Total SLOC	**28**	**45**	**38%**

Line counts are nearly identical for event logging itself, but the two implementations differ qualitatively. FOL's `:around` method is *declarative*—it applies automatically to all invocations, including future methods; Clojure's wrapper requires callers to remember to use the logged variant, and calling the multimethod directly silently skips logging. In mutable CLOS, both the `:around` interception and defensive copying of the event list would be needed to achieve the same guarantee.

The 38% reduction is the largest across our benchmarks. It is driven by declarative command routing via predicate specializers (`((cmd (deposit?)))`), native persistent updates (`assoc`, `conj`), and the elimination of separate `defstruct` types for commands. Persistence contributes a second guarantee: because `assoc` and `conj` produce new immutable values, the event log is append-only by construction—no code path can retroactively alter a recorded event, which is the core invariant event sourcing requires.

5.3 Observable Slots via MOP and Predicate Dispatch

This example combines persistent functional updates with predicate dispatch to build a change-notification system (repository file `observable.fol`). When a slot is updated, the protocol produces both a new object (via persistent `assoc`) and a change event; predicate dispatch routes notifications by content—e.g., detecting reading spikes (>50 unit jump) or critical status transitions:

```
(defclass <sensor> [<persistent-object>]
  [[name    :initarg :name]
   [reading :initarg :reading :initform 0]
   [status  :initarg :status  :initform :normal]])

(defn updated [obj slot new-val]
  (bind [old-val (get obj slot)
         new-obj (assoc obj slot new-val)]
    {:object new-obj
     :change {:object obj :slot slot
              :old old-val :new new-val}}))

(defn spike? [ch]
  (bind [change (get ch :change)]
    (and (= (get change :slot) :reading)
         (> (- (get change :new)
               (get change :old)) 50))))

(defgeneric on-change [change])
(defmethod on-change
  ([(c (spike?))]
   {:alert :spike
    :message (str "Spike detected: "
      (get (get c :change) :old) " -> "
      (get (get c :change) :new))})
([(c (critical?))]
 {:alert :critical
  :message "Sensor entered critical state"})
([c]
 {:alert :info
  :message (str "Slot " (get (get c :change) :slot)
    " changed")}))
```

Table 5: Code Comparison—Observable Slots

Category	FOL	CL	Savings
Class / boilerplate	6	23	74%
Update protocol	7	9	22%
Change dispatch	18	20	10%
Benchmark harness	14	0	—
Total SLOC	**45**	**52**	**13%**

The 13% reduction comes primarily from eliminating boilerplate: FOL needs no manual copy function (persistent `assoc` shares structure), no separate `defstruct` for change events (persistent maps suffice), and no package ceremony. This example best illustrates three-way synergy: persistence provides safe functional updates that produce change events without corrupting the original; predicate dispatch routes those events by content; and generic functions with `defmethod` enable new notification rules to be added without modifying the existing dispatch. None of these features alone suffices—Clojure has persistence but no method dispatch for extensible routing; CLOS has dispatch but requires manual deep copies to produce safe change events.

5.4 Lazy Schema Evolution

Standard CLOS handles class redefinition by lazily updating existing instances via `update-instance-for-redefined-class`: each instance is mutated in place the next time it is accessed. This protocol assumes mutable instances—an assumption FOL deliberately violates.

FOL's persistent metaclass resolves this tension through what amounts to *lazy schema evolution*. Class definitions remain mutable (redefining a class updates the class object normally), but existing instances are frozen snapshots whose native CLOS slots and overflow vectors are never modified after initialization. The MOP provides graceful degradation: when accessing a slot added by redefinition but absent from an old instance, the system falls back to the slot's `:initform` if one exists, or signals `slot-unbound` otherwise. Slots removed by redefinition remain in the instance but become inaccessible (no slot definition routes to them).

Crucially, functional updates perform implicit migration. `assoc` calls `(class-of object)` to obtain the *current* (redefined) class, allocates a fresh instance of that class, and shallow-copies the old slots forward. The resulting object is an instance of the new schema carrying the old data—new slots resolve via initforms, removed slots persist harmlessly. Persistence makes this *safe* in a specific sense: old instances are never corrupted and remain valid snapshots

of the prior schema; in mutable CLOS, redefinition silently mutates every reachable instance.

The tradeoff is that CLOS's `update-instance-for-redefined-class` provides an explicit migration hook where the programmer can transform slot values, while FOL's lazy approach silently returns initform defaults for new slots on old instances—a form of graceful degradation that could mask schema mismatches if the programmer is not aware. This lazy migration strategy is, to our knowledge, a novel adaptation of the MOP for immutable object systems—it trades explicit migration control for corruption safety and zero-downtime operation.

6 Evaluation

6.1 Comparison with Related Languages

Table 6: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Persistent objects	✓	✓	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	~[a]	×[b]
Transducers	✓	✓	×
Lazy seqs	✓	✓	×
Macros	✓	✓	✓

[a] `defmulti` dispatches on an arbitrary fn but lacks per-parameter specializers and specificity ordering.
[b] EQL specializers only; `optima`/`trivia` provide pattern matching as libraries.

FOL uniquely combines Clojure's functional features with Common Lisp's object system. Persistent structures incur $O(\log n)$ vs $O(1)$ for mutable operations, but high branching factors (32–64) keep constant factors small.

6.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Feature benchmarks report the mean of 100 runs (coefficient of variation <2%); LSim benchmarks report the mean of 10 runs. We compare against native Common Lisp rather than Clojure: SBCL compiles to native code, providing a strict upper bound on overhead, and since FOL transpiles *to* Common Lisp, this isolates abstraction cost. A Clojure comparison would conflate language design with JVM runtime differences.

6.3 Feature Benchmarks

We evaluated FOL across three benchmarks exercising specific language features: predicate dispatch (Trade Compliance), observability patterns (Sensor Observation), and event sourcing. Table 7 summarizes performance and code density.

Trade Compliance: This benchmark exercises conditional dispatch over persistent objects with 1,000,000 validation checks. The 2.16× slowdown stems from persistent allocation and Lisp-1 compatibility checks in the transpiled code. Memory allocation is 10× higher (1,164 MB vs. 113 MB) due to immutable map creation for each validation result.

Table 7: Feature Benchmark Summary

Benchmark	Perf Ratio	FOL	CL	SLOC %
Trade Compliance	2.16×	50	60	83%
Sensor Observation	5.88×	45	52	87%
Event Sourcing	8.10×	28	45	62%

Sensor Observation: Implements a monitoring loop with 1,000,000 iterations, producing multiple immutable event records per update. The 5.88× overhead reflects the cost of producing persistent maps for both the updated sensor state and the change event, with 6× higher memory allocation.

Event Sourcing: Leverages `:around` methods and persistent logs over 100,000 command applications. It shows the largest code density benefit (38% SLOC reduction) due to declarative command routing, with an 8.1× overhead for maintaining full persistent audit trails.

6.4 Logic Simulation (LSim)

LSim is a digital logic simulator that exercises persistent state management at scale. We evaluated three configurations: 8bit-100 (small), 32bit-300 (medium), and 8x32-900 (a 1024-gate pipeline of eight cascaded 32-bit registers). We also evaluated PLSim, a parallel variant that replaces the serial reduction over components with `preduce`, a parallel fold using a thread pool. Table 8 presents the results.

Table 8: LSim Scaling Results (Mean ms/run, 10 runs)

Config	CL	FOL	Ratio	PLSim	Ratio
8bit-100	1.0	11.3	11.3×	12.5	12.5×
32bit-300	8.4	58.1	6.9×	65.0	8.1×
8x32-900	444.9	1425.5	3.2×	1350.0	3.0×

The performance ratio narrows from 11.3× to 3.2× as circuit complexity grows, indicating that per-operation overhead is amortized over larger working sets. The parallel variant (PLSim) introduces thread-pool overhead that dominates at small scales (12.5× for 8bit-100), but begins to pay off for the 1024-gate pipeline, reducing the ratio from 3.2× to 3.0×. This crossover suggests that `preduce` becomes effective when the number of concurrently active components outweighs synchronization costs.

Beyond performance, FOL achieved a significant reduction in implementation effort, as shown in Table 9.

Table 9: LSim LOC Comparison

Module	FOL	CL	Ratio
LSim Core	217	293	0.74
8bit-100 test	67	79	0.85
32bit-300 test	158	220	0.72

The 15–28% SLOC reductions across the simulator core and test cases are primarily due to the elimination of CLOS boilerplate

(`defpackage`, `make-instance` wrappers, accessor `defun`s) and the use of concise collection literals for circuit definitions.

The PLSim parallel variant demonstrates that FOL's persistent data structures are naturally suited to parallel execution: because each component's state update produces a new immutable value, parallel reductions via `preduce` require no synchronization beyond the fold itself. The thread pool uses work-stealing to distribute component evaluations across available cores. At the 8x32-900 scale (~1024 gates, 900 time steps), the parallel overhead is amortized and PLSim outperforms serial FOL. We expect this advantage to increase further with larger circuits, as the ratio of useful computation to synchronization overhead improves.

6.5 Threats to Validity

All measurements compare FOL's transpiled code against SBCL's native compiler, so reported ratios are directly comparable. As a single-author project, FOL has not yet undergone independent code review; the full source is publicly available to facilitate external evaluation.

7 Limitations and Future Work

FOL is transpiled to Common Lisp. No static analysis exists—type errors and unreachable clauses are detected at runtime only; some type-dependent code may require runtime resolution, degrading transpiled performance. The compiler has not been extended to warn about shadowed clauses; a future version could detect shadowing statically.

Memory overhead grows with object size (1.1× for 2 slots, 4× for 100 slots). Random updates incur 21× overhead versus $O(1)$ mutation. Class redefinition does not migrate existing instances (Section 5.4); old objects retain their original slot data, with new slots resolved lazily via initforms. FOL's functional update pattern is related to **lenses** in the Haskell tradition; FOL does not yet provide a lens abstraction.

Planned extensions include **abstract and sealed classes**, **parallel collections** following Scala's model, and **channel-based concurrency** via CSP-style go blocks.

8 Related Work

Clojure [11] pioneered persistent data structures in Lisp. Clojure's *protocols* provide type-based polymorphism (single dispatch on the first argument's type) as a lightweight alternative to CLOS. FOL adopts Clojure's sequence abstraction and transducers but uses CLOS generic functions with multiple dispatch and method combinations—gaining `:before`/`:after`/`:around` qualifiers and MOP introspection at the cost of protocol performance optimizations.

Common Lisp's CLOS [3] and MOP [14] provide FOL's object system foundation, extended with persistent slots. **Dylan** [1] influenced FOL's `<type>` naming conventions. **Persistent data structures** build on Okasaki [15]; FSet [5] provides production-quality collections for CL. Clojure's `defrecord`, Scala case classes, and Kotlin data classes all provide value-oriented objects, but none integrate with a MOP—FOL is unique in combining persistent storage with metaclass-level extensibility.

Ernst et al. [9] introduced predicate dispatch with logical implication for specificity. Chambers and Chen [6] demonstrated efficient dispatch DAGs. **Filtered Dispatch** [16] extended CLOS with predicate-based method selection using explicit priorities. FOL's category-based specificity with definition-order tie-breaking provides a middle ground between logical implication (expressive but undecidable) and explicit priorities (simple but error-prone). **Scala's extractors** [8] and **Racket's match expanders** provide extensible pattern matching; FOL differs in that predicates are arbitrary functions and specificity is determined by category.

Julia [2] uses multiple dispatch but is purely type-based, lacking predicate specializers, method combinations, and MOP introspection. **Coalton** [17] adds Hindley-Milner types to CL—complementary to FOL (static safety vs. persistence). **Typed Racket** [20] adds gradual typing; FOL's type specializers serve as dispatch mechanisms rather than correctness guarantees. **Shen** [19] and **Carp** [18] offer pattern matching and static typing respectively, but lack multiple dispatch with method combinations. **Racket's class system** [10] provides method combinations but separates classes from `match`; FOL unifies pattern matching and method dispatch. **Elixir** combines pattern matching with protocols on the BEAM VM but lacks multiple dispatch and method combinations.

9 Conclusion

FOL demonstrates that persistent objects and CLOS are synergistic rather than competing paradigms. Their combination yields versioned objects via persistent snapshots, declarative event sourcing via method combinations over immutable logs, and observable slots via predicate dispatch over change events—patterns that require multiple features working together and are more natural than in either parent language alone. Lazy schema evolution further illustrates how immutability transforms a traditionally problematic MOP protocol into a corruption-safe migration strategy, albeit at the cost of CLOS's explicit migration hooks. Benchmarks show 2–11× overhead relative to hand-written Common Lisp, with the gap narrowing to 3× at scale and a parallel variant (`preduce`) demonstrating initial scaling benefits for large circuits. Across all benchmarks, FOL achieves 13–38% reductions in source lines of code. The complete implementation—over 600 exported functions, over 2,800 tests across 20 test suites, and approximately 18,000 lines of bootstrap code—is available at: https://github.com/frankadrian314159/fol

References

[1] Inc. Apple Computer. 1992. *Dylan - An Object-Oriented Dynamic Programming Language.* Apple Computer, Inc.

[2] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[3] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.

[4] Camillo Bruni. 2017. Fast Properties in V8. V8 Blog. https://v8.dev/blog/fast-properties

[5] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[6] Craig Chambers and Weimin Chen. 1999. Efficient Multiple and Predicate Dispatching. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications.* 238–255.

[7] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[8] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007–Object-Oriented Programming*. Springer, 273–298.

[9] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.

[10] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.

[11] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.

[12] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[13] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers

[14] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.

[15] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.

[16] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.

[17] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.

[18] Erik Svedäng. 2018. Carp: A Statically Typed Lisp Without a GC. https://github.com/carp-lang/Carp

[19] Mark Tarver. 2011. *The Shen Language*. Upaya Books.

[20] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.